Bitcoin Price Prediction Using High-Frequency Data

Naga Pujitha Pitta
20020425-7885
napi23@student.bth.se

Tharun Kumar Marada
2003126-3273
thma23@student.bth.se

Dhanush Kumar Majji
20020214-8219
dhma23@student.bth.se

Joshna Sindhuja Pitta
20030327-3023
jopi23@student.bth.se

Santhosh Birada
20030726-7955
sabi23@student.bth.se

***Abstract*—This project focuses on predicting short-term Bitcoin prices using high-frequency trading data. The data, collected over a six-hour period, was processed at one-minute intervals to reduce noise and highlight useful trends in price movement.**

To carry out the predictions, multiple models were developed and their performance was compared based on how accurately they could forecast the next price. The results of these models, including their prediction accuracy, were stored in a database for easy access.

A server application was built to fetch the results from the database, while a user interface was designed to display them in a clear and interactive manner. The overall system helps visualize how different models perform and serves as a foundation for building real-time crypto price analysis tools.

***Index Terms*—component, formatting, style, styling, insert**

I. Introduction

Cryptocurrencies have become a significant part of the global financial landscape, with Bitcoin leading the way as the most widely recognized and traded digital asset. Unlike traditional financial markets, the cryptocurrency market operates 24/7 and is known for its high volatility and rapid price movements. This makes the task of short-term price prediction particularly challenging, yet increasingly valuable for traders, analysts, and researchers.

As technology and data availability continue to improve, the use of data-driven approaches to understand and forecast market behavior has grown rapidly. High-frequency trading data, which captures market activity at very short intervals, presents new opportunities to explore short-term trends that may not be visible in daily or hourly price data.

The focus of this project is to analyze high-frequency Bitcoin data and explore how different models perform in predicting short-term price changes. By comparing the outcomes of multiple predictive approaches, the project aims to identify methods that can offer greater reliability in fast-moving market conditions.

The system developed as part of this project not only processes and analyzes the data but also presents the results through a user-friendly interface. This makes it easier to evaluate model performance and draw meaningful conclusions, setting the foundation for future applications in real-time cryptocurrency analytics.

II. Work Division

To ensure smooth progress and efficient execution of the project, responsibilities were divided among the team members based on their individual strengths and areas of interest. Each member contributed to a specific aspect of the project while maintaining coordination and collaboration throughout the development process. The table below outlines the primary roles and responsibilities undertaken by each team member during the course of the project.

Name	Responsibility
Naga Pujitha	Project coordination and documentation
Tharun Kumar	Data collection and preprocessing
Dhanush Kumar	Machine learning model development
Joshna Sindhuja	Database and system architecture
Santhosh	User interface and presentation design

III. Problem Description

A. Objective

The objective of this project are as follows:

1) To analyze high-frequency Bitcoin price data and re-sample it to a one-minute interval for short-term price prediction.
2) To implement and compare multiple predictive models including Linear Regression, Random Forest, XGBoost, and LSTM.
3) To evaluate each model's performance using standard regression metrics and store the results in a structured database.
4) To design a user-friendly interface for visualizing model performance and displaying actual vs predicted Bitcoin prices in real-time and evaluation metrics.

B. Stake Holders

1) *Traders*: Can gain insights from short-term price forecasts to support trading decisions.
2) *Analysts and Researchers*: May use the model outputs to study market behavior and evaluate forecasting techniques.

3) *Developers*: Can build on this architecture to create more advanced systems for real-time analysis.
4) *Students and Educators*: May use the system as a learning resource or project reference for topics related to machine learning, finance, and software development.

IV. PROJECT ARCHITECTURE

The architecture of the project is designed as a modular pipeline that efficiently handles high-frequency Bitcoin data for short-term price prediction. The system begins with the processing of raw tick-level data, which is resampled to a one-minute interval and enriched with engineered features such as mid price, spread, and lag values. This processed data is then used by the machine learning module to train multiple predictive models, including Linear Regression, Random Forest, XGBoost, and LSTM. Each model generates predictions and is evaluated using standard metrics such as RMSE, MAE, and R^2. The resulting predictions and metrics are stored in a centralized SQLite database. A Flask-based server module retrieves this data on request and serves it to the front-end interface. The user interface, built using Streamlit, fetches data from the server and displays it through interactive graphs and tables, allowing users to compare model performance in a visually intuitive manner. This architecture ensures a clear separation of responsibilities across modules while maintaining a smooth flow of data from input to output.

A. Problem-Proposed Solution

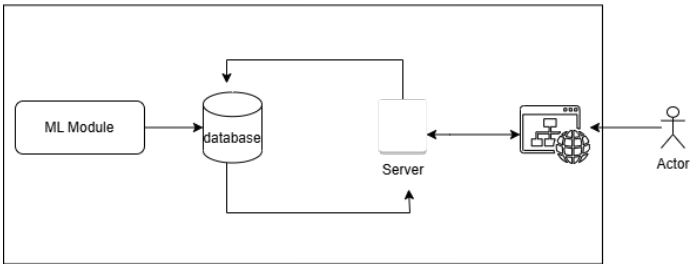


Fig. 1. Project architecture

1) *Database Module*: The database serves as the central storage system for the application. It stores model predictions and performance metrics in a structured manner. SQLite is used as the database solution due to its lightweight nature and ease of integration. All data generated by the machine learning models is inserted into the database, which is later accessed by the server module to serve UI requests.
2) *Data Processing Module*: This module is responsible for preparing the high-frequency raw data for modeling. The original dataset contains over 17 million entries recorded within a six-hour timeframe. The data is resampled to a one-minute interval to reduce noise and highlight meaningful trends. This module also performs essential feature engineering tasks such as the calculation of mid price, spread, order imbalance, momentum, and lag features.
3) *Machine Learning Module*: In this module, various predictive models are implemented and trained to forecast the short-term price of Bitcoin. The models used include Linear Regression, Random Forest, XGBoost, and LSTM. Each model is evaluated based on metrics like RMSE, MAE, and R^2. The predictions and evaluation results are stored in the database for retrieval. The training process is optimized to run efficiently, with multiple models being trained in parallel where applicable.
4) *Server Module*: The server module acts as a bridge between the database and the user interface. Built using Flask, it exposes API endpoints that the UI can call to fetch model metrics and prediction results. This module ensures secure and structured communication between the data storage and presentation layers, making it possible to build a responsive and interactive front end.
5) *UI Module*: The user interface is developed using Streamlit to provide a clean and interactive dashboard for users. It displays a comparison table of model metrics and line plots showing actual vs predicted prices for each model. The interface is designed to be intuitive, allowing users to analyze model performance visually and in real-time by interacting with the displayed data.

V. METHODOLOGY

The methodology followed in this project is centered around building a reliable and modular machine learning pipeline for short-term Bitcoin price prediction. Starting with high-frequency raw data, each step in the pipeline—from preprocessing and feature engineering to training, evaluation, and visualization—was carefully designed to ensure meaningful results. The data was cleaned, transformed, and enriched to make it suitable for training predictive models. A one-minute time frame was adopted to reduce noise while retaining important patterns. Multiple models were trained on this data and evaluated using standard metrics to determine their effectiveness.

1) *Data collection*: The dataset used in this project was

id	symbol	sequence	ask_price	ask_size	bid_price	bid_size	time_coinapi	time_exchange	ts
fdc75135-e60f-4f12-9c51	KUCOIN_SPOT_DOGE_KCS	11486	0.006679	18272.3495	0.006665	3380.6599	05:40.1	05:40.1	05:48.6
07a2b1e0-0da2-4527-aea	BINANCEFTS_PERP_RUNE_USDT	48137	2.864	11603	2.863	4023	05:48.5	05:48.4	05:48.6
75e4c1b4-98a9-497c-bd2	OKEX_SPOT_BCH_USDT	36271	140.41	0.789202	140.4	0.071009	05:48.3	05:48.3	05:48.7
00245367-0e1b-466c-a35	BINANCE_SPOT_TRB_USDT	48881	40.82	2.92	40.79	2.96	05:42.1	05:42.1	05:48.7
7e118fc3-51d1-4cc7-a0f	BINANCEFTS_PERP_WAVES_USDT	55843	6.002	4575.4	6.001	254.9	05:48.5	05:48.4	05:48.7
923d089b-e918-474a-af7	BINANCE_SPOT_LRC_ETH	5300	0.00024254	238	0.0002422	503	05:42.1	05:42.1	05:48.7
8f035eed-25d1-4ab4-8bc	BINANCEFTS_PERP_LUNA2_BUSD	28176	2.0351	2534	2.035	3	05:48.5	05:48.4	05:48.7
bde72c87-b209-4957-8c6	BINANCE_SPOT_OP_USDT	162075	1.577	32.99	1.576	15851.33	05:42.1	05:42.1	05:48.7
2586a728-92d3-4282-a1f	BINANCE_SPOT_ETH_BUSD	308410	1892.61	17.8526	1892.41	2	05:42.1	05:42.1	05:48.7
eb490aba-4236-478b-b25	KRAKEN_SPOT_BTC_USD	123932	23999.7	3.12503906	23999.1	0.00443493	05:48.6	05:48.6	05:48.7
558570fa-ebce-4535-9f3c	KRAKEN_SPOT_BTC_USDT	93056	23991	0.62503873	23986.3	0.13759952	05:48.6	05:48.6	05:48.7
b81ca865-c460-4ba7-911	KRAKEN_SPOT_BTC_USDT	93057	23991	0.62503873	23988.8	0.62509358	05:48.6	05:48.6	05:48.7
63c263ee-889a-4fcd-b81c	KRAKEN_SPOT_BTC_USD	123933	23999.7	3.12503906	23997.5	3.12531715	05:48.6	05:48.6	05:48.7
e220f50e-f511-43d1-b2a0	KRAKEN_SPOT_BTC_USDC	77811	24004.07	0.05358893	23989.3	0.08567736	05:48.6	05:48.6	05:48.8
ef511221-40e4-4913-86e	KRAKEN_SPOT_BTC_USD	123934	23999.7	3.12503906	23999.1	0.00443493	05:48.7	05:48.7	05:48.8
2c48bc55-915b-4048-b07	KRAKEN_SPOT_BTC_USDT	93058	23991	0.81503873	23988.8	0.62509358	05:48.7	05:48.7	05:48.8
27890a53-d5c3-4879-986	KRAKEN_SPOT_BTC_USDC	77812	24004.07	0.05358893	23990.49	0.125	05:48.8	05:48.8	05:48.8
efdfbf3c-4cd8-4b14-9716	KRAKEN_SPOT_BTC_USDC	77813	24004.07	0.05358893	23990.5	0.04167361	05:48.9	05:48.9	05:49.0
bdd7c4bb-ffce-7-473a-89d	KRAKEN_SPOT_BTC_USD	123935	23999.7	3.22503906	23999.1	0.00443493	05:49.0	05:49.0	05:49.0
7362c251-dfbe-420b-9a4	KRAKEN_SPOT_BTC_USD	123936	23999.7	3.22503906	23997.5	3.12531715	05:49.0	05:49.0	05:49.0
6be46910-a4e9-40eb-b4e	KRAKEN_SPOT_BTC_USD	123937	23999.7	3.22503906	23998.9	0.00443493	05:49.0	05:49.0	05:49.1
c28140d6-5b7f-42fd-b42f	KRAKEN_SPOT_BTC_USD	123938	23999.6	0.6007353	23998.9	0.00443493	05:49.1	05:49.1	05:49.1
cb96c100-e8ca-4603-884	KRAKEN_SPOT_BTC_USDC	77814	24004.07	0.05358893	23991.69	0.125	05:49.1	05:49.1	05:49.1
3a5ea92c-a820-46ce-a5c	KRAKEN_SPOT_BTC_USDT	93059	23991	0.62503873	23988.8	0.62509358	05:49.2	05:49.2	05:49.2
6a59fc4a-b645-4566-8b2	KRAKEN_SPOT_BTC_USDC	77815	24004.07	0.05358893	23991.7	0.04167361	05:49.9	05:49.9	05:50.0
f9b7b0c8-f3a4-48e8-9e1	KRAKEN_SPOT_BTC_USD	123939	23999.6	0.6007353	23999	0.30979799	05:50.0	05:50.0	05:50.0

Fig. 2. Dataset

provided by our professor and contains high-frequency Bitcoin trading data captured over a period of six hours. The dataset originally consisted of over 17 million rows, with each row representing a single entry of market

activity. The key features included in the dataset are as follows:

- *id* – A unique identifier for each entry
- *symbol* – The cryptocurrency symbol (e.g., BTC)
- *sequence* – An internal sequence number
- *ask_price, ask_size* – The lowest price sellers are willing to accept and the size of that offer
- *bid_price, bid_size* – The highest price buyers are willing to pay and the size of that bid
- *time_coinapi, time_exchange* – Timestamps from the data provider and exchange respectively
- *ts* – A standard timestamp used for alignment and resampling

Some of these columns were not relevant to our prediction task and were removed in the preprocessing stage.

2) *Data preprocessing*: To make the dataset more manageable and suitable for modeling, several preprocessing steps were applied. First, irrelevant columns such as id, sequence, and duplicate timestamps were removed. The raw data had over 17 million entries within a span of six hours, making it highly granular and noisy. To address this, we resampled the data to a 1-minute interval by taking the average of price-related features within each minute. This drastically reduced the dataset size to around 20,000 entries, while still retaining essential trends and fluctuations. Missing or incomplete rows were also removed to ensure clean input for the modeling process.
3) *Feature engineering*: After resampling, we created additional features to help the models learn more effectively. These include:
   - *Mid Price*: The average of the bid and ask prices
   - *Spread*: Difference between the ask and bid prices
   - *Order Imbalance*: Ratio of bid size minus ask size to the total of both sizes
   - *Momentum*: The difference in mid price over past intervals
   - *Rolling Mean & Standard Deviation*: To capture local trends and volatility
   - *Bollinger Bands*: Upper and lower bands based on rolling statistics

   We also generated lag features, where we used the values of the previous 60 time steps (1-hour worth of data at 1-minute resolution) as inputs for predicting the next mid price. These lag features helped traditional ML models recognize time-based patterns without having built-in sequence awareness.
4) *Data splitting*: The final dataset was split into training and testing sets using an 80/20 time-based split. Since the data is sequential, shuffling was avoided to maintain temporal integrity. For each training instance, we used the previous 60 entries as input features to predict the next value of the mid price. This approach was used consistently across all models to ensure a fair and meaningful comparison of their performances.

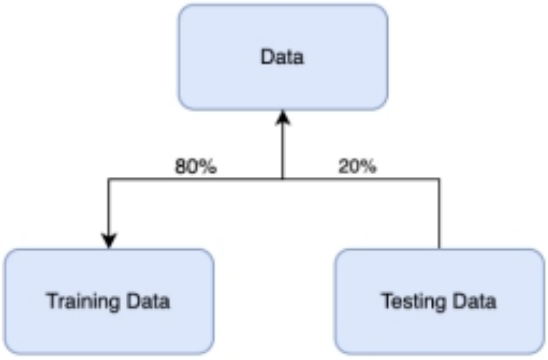


Fig. 3. Data split

5) *Model selection*: To compare different approaches for short-term price prediction, we selected a combination of both traditional machine learning models and a deep learning model. The models used were:
   - **Linear Regression**: A simple and interpretable baseline model
   - **Random Forest Regressor**: A tree-based ensemble model that handles non-linear relationships well
   - **XGBoost Regressor**: A highly efficient gradient-boosted tree algorithm known for its performance in structured data tasks
   - **LSTM (Long Short-Term Memory)**: A recurrent neural network capable of learning from sequences and capturing temporal dependencies in data

   This mix of models was chosen to provide a balanced comparison between conventional tabular data methods and sequence-based deep learning techniques.
6) *Model Training*: Each model was trained on the processed dataset using the same training set for fairness. For traditional models (Linear Regression, Random Forest, and XGBoost), the 60 previous entries were used as lagged input features for each prediction. The LSTM model, on the other hand, was trained using sequences formatted as time-series input, where the input shape was 60 time steps with multiple features per step.
   Hyperparameters for each model were chosen based on prior experimentation and best practices, ensuring stable training without overfitting. The training process was executed in a modular setup, allowing each model to run independently, and in some cases, in parallel to optimize development time.
7) *Model Testing*: After training, all models were evaluated using the same test dataset (the remaining 20% of the data). Predictions were generated using the final trained models and compared against actual mid prices. For the LSTM model, the sequence structure of the test set was maintained, while for traditional models, the test data was

structured using the same lag-based format as used during training.

This testing phase allowed us to observe how well each model generalized to unseen data and how their predictions performed in terms of accuracy and trend alignment.

8) *Model Evaluation*: To assess the performance of each model, we used three standard regression metrics:
   - **Root Mean Squared Error (RMSE)**: Measures the average magnitude of the prediction errors
   - **Mean Absolute Error (MAE)**: Represents the average absolute difference between actual and predicted values
   - R^2 **Score (Coefficient of Determination)**: Indicates how well the predictions approximate the actual values

   The predicted values and their corresponding evaluation metrics were stored in a SQLite database. These results were later retrieved through a Flask-based server and displayed using a Streamlit interface, allowing for side-by-side comparison of model performance through both tables and graphs.

VI. SWOT ANALYSIS

A. Strengths

- Utilizes high-frequency data, which enables more accurate short-term price prediction.
- The system follows a modular architecture, making each component easy to manage, update, and test independently.
- Supports multiple machine learning models, allowing for comparison and flexibility in improving prediction performance.
- The user interface is clean and interactive, making it easier for users to interpret the output visually.

B. Weaknesses

- The dataset covers only six hours of trading data, which may limit the model's ability to generalize over longer periods.
- Training the LSTM model takes more computational time and resources compared to traditional models.
- The project is limited to Bitcoin and does not currently support predictions for other cryptocurrencies.

C. Opportunities

- The system can be extended to support real-time data streaming and live price prediction.
- The current framework can be adapted to include multiple cryptocurrencies or financial instruments.
- Incorporating additional data sources like social media sentiment or financial news could improve prediction accuracy.

D. Threats

- The unpredictable and volatile nature of the cryptocurrency market may impact model performance.
- Sudden market events or structural changes may not be captured well by models trained on historical data.
- Scaling the system for real-time predictions or more currencies may require additional infrastructure and optimization.

VII. RESULTS

A. User Interface (UI)

The user interface was developed using Streamlit, a Python-based framework designed for building interactive data applications. Its purpose is to allow users to easily view and compare the performance of different machine learning models used for predicting short-term Bitcoin prices.

The interface contains two main components. The first is a table that displays the evaluation metrics of each model, including RMSE, MAE, and R² scores. This table is generated by fetching data from a Flask server, which retrieves the results from a centralized SQLite database.

The second component consists of a set of line graphs that show the actual versus predicted Bitcoin prices for each model. These graphs are arranged in a 2x2 grid layout, making it easy to view and compare the performance of all four models at once. When a user opens the interface, it automatically calls API endpoints for each model to retrieve prediction data and display the graphs.

Overall, the user interface provides a clean and intuitive way to understand how each model performs, without needing to look at raw numbers or code.

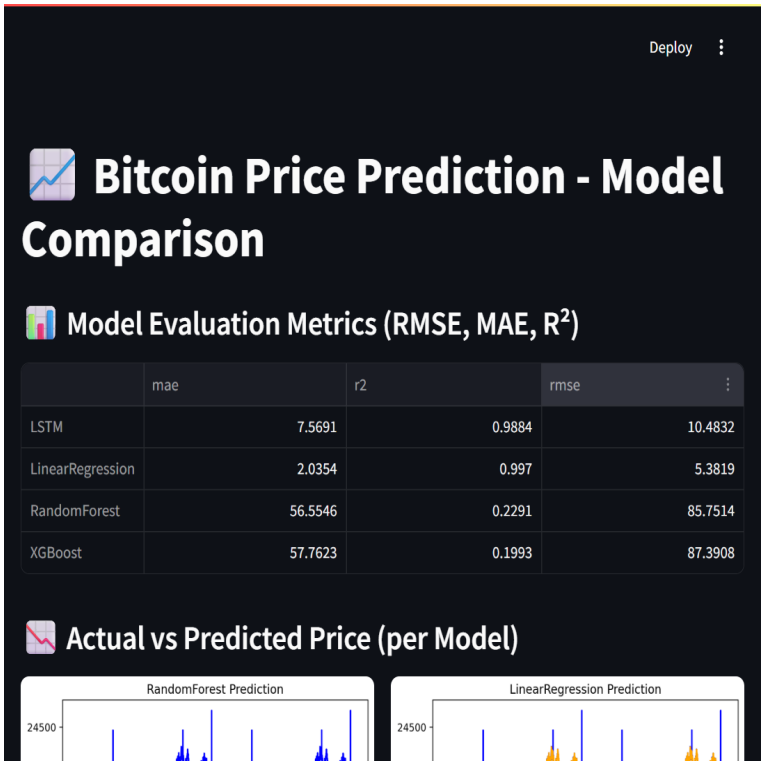


Fig. 4. Metrics

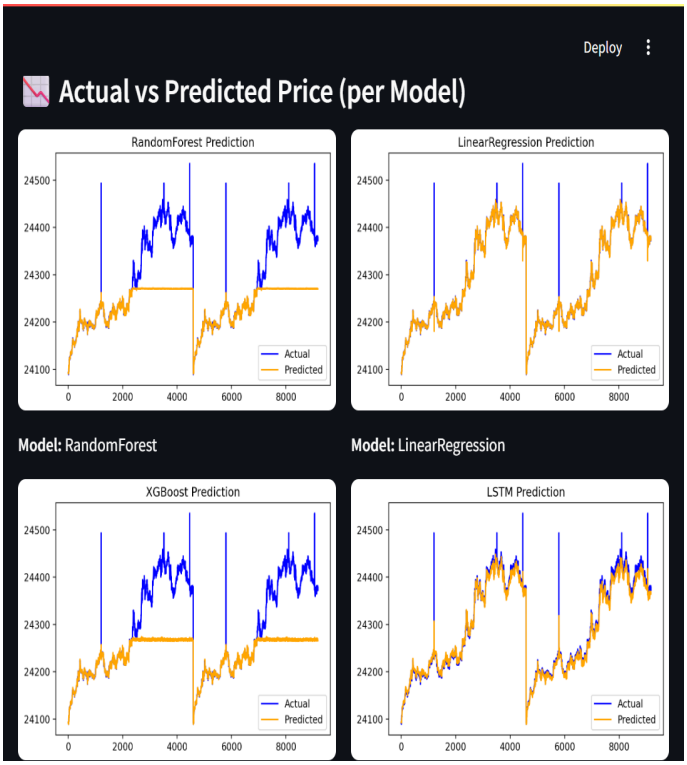


Fig. 5. prediction graphs

B. Metrics analysis:

To compare the performance of the models used in this project, three widely accepted evaluation metrics were used: Mean Absolute Error (MAE), Coefficient of Determination (R^2), and Root Mean Squared Error (RMSE). These metrics were calculated on the test data and stored in a database for easy retrieval and visualization. Each metric offers a different perspective on model performance, helping us understand both accuracy and reliability in predicting short-term Bitcoin prices. The results for each model are discussed below.

1) **Mean Absolute Error (MAE)**
   Mean Absolute Error measures the average of the absolute differences between the predicted and actual values. A lower MAE indicates better predictive accuracy.
   - **Linear Regression** achieved the lowest MAE of 2.0354, suggesting that its predictions were very close to actual values on average.
   - **LSTM** also performed well, with an MAE of 7.5691, which is reasonable given its sequential learning approach.
   - **Random Forest and XGBoost** on the other hand, had significantly higher MAE values of 56.5546 and 57.7623 respectively, indicating that their predictions were less accurate on average.
2) **Coefficient of Determination (R^2)**
   R^2 represents how well the predictions approximate the actual values. A value closer to 1 indicates a better fit.
   - **LSTM** performed best in this category, achieving an R^2 score of 0.9884, meaning it captured the variance in the data quite effectively.
   - **Linear Regression** also showed strong performance with an R^2 score of 0.997, the highest among all models.
   - In contrast, **Random Forest and XGBoost** scored 0.2291 and 0.1993 respectively, suggesting that these models were unable to capture the underlying data patterns effectively.
3) **Root Mean Squared Error (RMSE)**
   RMSE provides a measure of the average magnitude of the prediction errors, with larger errors penalized more than smaller ones.
   - **Linear Regression** again performed best with the lowest RMSE of 5.3819, indicating more consistent predictions.
   - **LSTM** followed with an RMSE of 10.4832, which is still within an acceptable range for this task.
   - **Random Forest and XGBoost** had much higher RMSE values of 85.7514 and 87.3908 respectively, confirming the less reliable performance seen in other metrics.

Based on the evaluation metrics, Linear Regression emerged as the best-performing model across all three categories—achieving the lowest MAE and RMSE, and the highest R^2 score. LSTM also showed strong results and was able to effectively model the time-series nature of the data. In contrast, Random Forest and XGBoost performed poorly, likely due to their limited ability to handle the sequential structure of the high-frequency data without explicit time-based features. These results highlight the importance of choosing the right model architecture based on the nature of the data.

VIII. CONCLUSION

This project aimed to develop a system capable of predicting short-term Bitcoin prices using high-frequency trading data. By resampling the raw data to a one-minute interval and applying various feature engineering techniques, we created a dataset suitable for time-series forecasting.

A range of predictive models, including Linear Regression, Random Forest, XGBoost, and LSTM, were implemented and compared based on their performance. The predictions and evaluation metrics were stored in a database and made accessible through a user-friendly interface that allowed for visual and tabular comparison of results.

The system demonstrated the ability to process, model, and present high-frequency data effectively. While limited to Bitcoin and a short time window, the project successfully achieved its core objective of building a functional and modular prediction pipeline. It also laid the groundwork for future extensions, including multi-currency support and real-time prediction capabilities.

IX. FUTURE WORK

While the current system provides a strong foundation for short-term Bitcoin price prediction, there are several areas

where the project can be further developed and enhanced. One potential improvement is to extend the model to support real-time data integration and live predictions, allowing users to observe model output in near real-time.

Another direction is to include additional cryptocurrencies in the analysis. Expanding the system to predict the prices of other major digital assets would increase its usefulness and test the scalability of the current architecture. Incorporating external data sources, such as market sentiment from social media or real-time news feeds, could also improve prediction accuracy by capturing market behavior that price data alone may not reflect.

From a technical standpoint, the system could be optimized to handle larger volumes of data more efficiently. Implementing database optimizations, model retraining schedules, and performance tuning would make it more suitable for production use. Lastly, exploring more advanced deep learning architectures, such as transformers or hybrid models, could further enhance the predictive capability of the system.

X. REFERENCES

REFERENCES

[1] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[2] T. Chen and C. Guestrin, "XGBoost: A Scalable Tree Boosting System," in Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[3] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," [Online]. Available: https://bitcoin.org/bitcoin.pdf, 2008.

[4] Y. Fang, J. Zhang, and J. Wang, "Price Prediction of Bitcoin using LSTM and Sentiment Analysis," in 2022 International Conference on Artificial Intelligence and Computer Engineering (ICAICE), pp. 192–196.

[5] J. Brownlee, "Time Series Forecasting with Python," Machine Learning Mastery, 2018. [Online]. Available: https://machinelearningmastery.com/time-series-forecasting/